ClubPulse website guide

Product walkthrough and launch instructions

Prepared for the club project team and hackathon presenter
24 October 2026

ClubPulse lets participants submit event feedback with a one-use code, then gives authorised organisers a protected event report and tools to plan improvements. This guide explains the current website, how its privacy design works, how to run it, and the steps required before a public club launch.

Current status: the local preview and production build are available. No GitHub push, public deployment or custom-domain connection has been completed. The application still needs production infrastructure verification. This document distinguishes implemented features from future work.

Read this first

The website has two entry points. Participants use event codes and never receive administrator privileges. Administrators use email and password to manage events, issue invitations, and read released reports. Participant sign-in verifies event eligibility without creating a named account.

Contents

2 Website pages and access
3 Participant and administrator workflows
4 Statistics and organiser tools
5 Anonymity and data architecture
6 Local setup and troubleshooting
7 Production configuration
8 GitHub and launch checklist
9 Testing and operation
10 Hackathon demonstration and next steps

Design and demo data

The interface uses paper tones, black typography, yellow highlights, red accents and square controls. It includes a favicon and locally hosted fonts, with no AI badge or third-party trackers. The preview has one 24-Hour Hackathon and five explicitly synthetic ratings: 5, 4, 4, 3 and 5. Their frozen average is 4.2 out of 5. New test submissions do not change that fixture.

Website pages and access

Page or route	Who uses it	Purpose
/	Everyone	Event discovery and separate sign-in entrances.
/#participant	Participants	Select an event and exchange an invitation code.
/#participant/EVENT_ID	Participants	Open the selected event directly.
/#proof	Everyone	Read the anonymity flow and its limitations.
/#admin/login	Administrators	Email and password sign-in.
/#admin/statistics	Administrators	Sealed reports, charts, comparisons and action board.
/#admin/feedback	Administrators	Search already released, shuffled comments.
/#admin/events	Administrators	Create events, issue invitations and close collection.
/#admin/privacy	Administrators	Review privacy rules and deployment cautions.

The security boundary

A hidden button is not an access control. Each administrator API checks a valid server session with the admin role. Opening an administrator URL without signing in displays the sign-in form. Participant codes, signed participant tokens and fabricated session cookies do not grant administrative access.

The app has one configured administrator credential pair, not a multi-user organisation directory. There is no self-service admin registration, password reset, administrator invitation system or MFA in the current implementation. Manage credentials through the deployment environment.

Shared computer rule

Sign out of the administrator workspace before giving the device to a participant, or use separate browser profiles. A browser already signed in as administrator retains that access. Participant forms omit account cookies when sending invitations and feedback; that protects the submission flow, not a shared unlocked browser.

Public metadata

Visitors can see event names, descriptions, dates and open status. Public event endpoints do not return feedback, reports, private signing keys or response counts.

Participant and administrator workflows

Participant journey

1. Obtain the event link and one unique invitation code from the organiser. Codes are distributed manually; the app does not send email or SMS.

2. Open Participant sign-in, select the event and enter the code. In local preview, Use demo code only fills the field. Nothing is transmitted until Sign in with event code is pressed.

3. The browser creates a random token and blinds it. The issuer receives the invitation code and blinded token, validates the code and signs the blinded value. The browser unblinds the signature.

4. Choose a rating from 1 to 5, select Content, Delivery, Organisation or Other, and optionally write up to 2,000 characters. Remove names, team names and identifying details.

5. Submit. The collector receives the event, rating, topic, comment and signed token. It does not receive the invitation code, account cookies or referrer. Each valid token can be accepted once.

The browser keeps an unfinished token exchange in session storage for retry. Losing that storage may lose access to an already exchanged invitation. There is no email recovery because no participant email is collected. There is currently no participant reply inbox or receipt lookup.

Administrator journey

1. Sign in, open Events, and create the event with a name, date and description. New events accept feedback immediately.

2. Open Manage invitations. Generate 1 to 200 codes per request and download them immediately. Share one code per attendee. Plaintext codes are not retrievable later from the database.

3. Copy the event-specific participant link. Keep the administrator password and the full invitation list private. Avoid keeping an identity-to-code mapping if your club does not need one.

4. Close the event when collection finishes. Real reports require at least five responses and a full 15-minute cooldown after closure. Exact collection counts remain hidden.

5. Refresh Statistics after eligibility. The first eligible report request seals the report. A real event with a sealed report cannot reopen. Create a new event for a new cohort.

6. Review released comments, choose an organiser action, assign a responsible role and due date, and export the aggregate brief. Sign out when finished.

Statistics and organiser tools

Read the report

The report scope selects all sealed reports or one event. The overall score is a response-weighted average rounded to one decimal. The displayed response total is not attendance, a participation rate or a count of unique people. Someone receiving several valid codes could submit several responses.

Positive means rating 4 or 5. The positive percentage is released only if both the positive and non-positive groups contain at least five responses. With the five demo ratings, this metric stays protected. The 5+ voices label indicates the release minimum, not a positive percentage.

Explore without exposing participants

Rating bars support Count and Percent modes. The entire rating or topic breakdown remains withheld if any bucket or its complement has fewer than five responses. Changing units never bypasses suppression. Expand What these numbers mean for definitions. The release ledger shows collection, cooldown and report status.

Compare events accepts two real sealed reports and shows their rounded mean difference and cohort sizes. Demo reports are excluded. The comparison is descriptive; it does not establish statistical significance, participant-level change or causation. With only one synthetic event, the comparison explains how to unlock it rather than inventing a baseline.

Turn comments into work

Listening notes use simple keyword rules on released comments to suggest practical improvements. Review the source comments before acting. They are not AI sentiment analysis or measured prevalence. The released-comment search only searches text already present in the administrator browser.

The action board supports custom actions, responsible organiser roles, due dates, Planned / In progress / Done status, a completion bar, suggestions from listening notes, removal with undo, and JSON download. Each report scope has its own board. A separate free-text action-plan note is also retained.

Storage limit: action boards and notes are saved only in the current browser. They are not shared between administrators and can be lost when browser data is cleared. Download private action files when needed. Do not add participant identities.

Export options

Control	Output	Contains
Export CSV	CSV file	Sealed event aggregates only.
Download brief	HTML file	Readable aggregate report; open and print to PDF.
Copy the takeaways	Clipboard text	Summary and optional free-text plan; review before sharing.
Download actions	JSON file	Private organiser tasks, roles, dates and statuses.

Anonymity and data architecture

Two services with different responsibilities

React runs in the browser. The Express collector serves the website, handles administrator sessions and verifies anonymous feedback. A separate issuer owns invitation hashes and private event signing keys. MongoDB persists the two services in separately scoped databases. Local demo listeners share this computer and process; production needs separate HTTPS origins and service processes.

Browser → issuer: invitation code + blinded token.
Issuer → browser: blind signature.
Browser → collector: feedback + unblinded token + signature.
Administrator → collector: authenticated event management and sealed reports.

The implementation uses Cloudflare blindrsa-ts and the RFC 9474 RSABSSA-SHA384-PSS-Randomized suite, with a 2048-bit RSA key per event. The collector holds only public keys. Canonical token hashes and database uniqueness reject replay attempts, including alternative Base64 formatting.

Location	Stored information
Issuer database	Event private/public keys; invitation hashes; blinded-request hashes used for exchange retry protection.
Collector database	Event metadata and public keys; random feedback UUID, event ID, rating, topic, comment and unique token hash; sealed reports.
Server memory	Administrator sessions and temporary rate-limit keys derived from network addresses.
Participant browser	Pending exchange and signed token in session storage until completion.
Administrator browser	Action notes and board tasks in local storage.

What is deliberately absent

Feedback records contain no name, email, invitation code, IP address, user-agent or submission timestamp. Released comments are shuffled and detached from rating, topic, row ID and time. Reporting APIs permit whole-event scope only. Reports are fixed after release to reduce differencing attacks.

What anonymity does not guarantee

Hosting providers can observe network addresses and timing. Operators with broad infrastructure access may correlate traffic. Database access can expose raw feedback. Identifying comments can reveal their author. Threshold rules are not differential privacy and do not stop all inference. The complete application has not had an independent security audit.

Local setup and troubleshooting

Start the temporary preview

Install Node 24, open PowerShell in the project directory, and run the following commands. The current working preview may already be running; do not start another process on the same port.

```
cd C:\Users\2026\Documents\ChatGPT\clubpulse
npm ci
npm run dev
```

Open http://localhost:3000. The collector listens on port 3000 and the demo issuer on 3001. Without a database URI, local development uses temporary memory. Restarting clears events and submissions, recreates the fixture and invalidates administrator sessions.

Local preview item	Value
Administrator email	admin@clubpulse.local
Administrator password	local-preview-only
Participant demo code	HACKATHON-DEMO-2026

These are local demonstration credentials only. Production forbids demo mode. The demo code works once per restart; generate fresh invitations to test more participants.

Use persistent databases

1. Create collector and issuer databases with separate users. Collector transactions require a MongoDB replica set, such as an appropriately configured Atlas cluster. A standalone MongoDB server is insufficient.

2. Copy .env.example to .env and .env.issuer.example to .env.issuer. Fill in the database URIs and secrets. Keep the files private and out of Git.

```
Copy-Item .env.example .env
Copy-Item .env.issuer.example .env.issuer
node --env-file=.env.issuer server/issuer.js
```

3. Leave the issuer terminal running. In a second terminal, run npm run dev. With persistent configuration, create real test events; the synthetic fixture is disabled.

Common problems

Code already exchanged: resume in the same tab if its token remains. Otherwise ask the organiser for a replacement. Protected report: check event closure and the full cooldown; fewer than five responses stay private. Login lost after restart: expected because sessions are in memory. Port already in use: stop the existing project process or choose consistent collector and issuer ports. Issuer request fails: check its service health, origin configuration and shared secret.

Production configuration

Deploy two Node web services and persistent MongoDB databases. Static-only hosting cannot run the backend. Choose hosting and a domain you control, review the provider billing screen, and configure the following values. No purchase has been made by this project.

Collector environment	Required value
NODE_ENV / DEMO_MODE	production / false
MONGODB_URI	Collector-only database URI with replica-set support.
ADMIN_EMAIL / ADMIN_PASSWORD	Your private administrator credentials; password at least 12 characters.
SESSION_SECRET	Independent random secret, at least 32 characters.
ISSUER_ADMIN_SECRET	Random shared service secret, at least 32 characters.
PUBLIC_ORIGIN	Exact HTTPS website origin, no trailing slash.
ISSUER_URL	Different HTTPS issuer origin, no trailing slash.

Issuer environment	Required value
NODE_ENV / DEMO_MODE	production / false
ISSUER_MONGODB_URI	Issuer-only database URI with its own database user.
ISSUER_ADMIN_SECRET	Exactly the same service secret as the collector.
PUBLIC_ORIGIN	Exactly the collector website origin.

Build and start commands

Collector: build with npm ci followed by npm run build; start with npm start. Health endpoint: /api/health. Issuer: build with npm ci; start with npm run issuer. Health endpoint: /health. Let the host supply PORT for each service.

Generate secrets locally

```
node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"
```

Run this separately for each independent secret. Never copy real values into screenshots, this document, source code or a public issue. Match only the service-to-service secret across the two services.

Infrastructure requirements

Use one collector instance until shared session storage is added. Restrict database users and network access. Configure host, proxy and APM logging to avoid sensitive payloads. Test both HTTPS origins, CORS, TLS and persistence on the actual host. Separate origins alone do not prevent a common operator from correlating traffic.

GitHub and launch checklist

Approve and publish the source

Review the local preview before pushing. Create an empty GitHub repository named clubpulse. Replace the repository URL below with the URL from your own account. These commands have not been executed for you.

```
git init
git add .
git status
git diff --cached --stat
git commit -m "Build ClubPulse event feedback platform"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/clubpulse.git
git push -u origin main
```

Before the commit, verify that .env, .env.issuer, invitation downloads, database files and private keys are not staged. Example environment files may be included. Use the existing remote if the project is already a repository; do not add a duplicate origin.

Connect the custom domain before launch

1. Configure both Node services with the production environment from page 7. Set their exact final HTTPS origins.

2. Add your website domain and issuer hostname in the hosting dashboard. Copy the DNS records supplied by that host to your domain provider. DNS records are provider-specific and cannot be invented in advance.

3. Wait for domain verification and valid TLS certificates. Confirm both services are reachable without certificate warnings. Restrict default host URLs where supported.

4. Open the website on its custom domain. Confirm the favicon loads, demo mode is off and no hosting provider injects a Made with AI badge. Check desktop, phone, keyboard navigation and sign-in.

5. Run the real privacy and persistence checks on page 9. Confirm the website calls the correct issuer origin and sends no cookies or referrer on anonymous submissions.

6. Only after these checks, submit the custom-domain link and repository link. Keep a local backup demonstration in case the venue network fails.

Cost and account decisions

No domain or hosting plan has been purchased. Free plans may have constraints, and domain registration may cost money. Verify the selected provider terms and spending limits before activating a plan. This guide does not promise free hosting or account-specific billing eligibility.

What remains outside the code

You must supply deployment accounts, database access, production secrets and domain DNS access. Infrastructure configuration, provider logs, backup restore and live database persistence remain unverified until checked on those accounts.

Testing and operation

Run the project checks

```
npm test
npm run build
```

The existing seven-test suite covers separate issuance, credential-free collection, admin route protection, forged signatures, replay, forbidden identity fields, cooldown, immutable reports, small-cell suppression, detached comments and aggregate-only export. It uses isolated temporary services and an accelerated test-only cooldown. Production keeps the full 15-minute wait.

Acceptance checks on your deployment

1. Without signing in, request statistics, exports, event creation, event updates and invitation generation. Each must require administrator authentication. A participant token must not grant admin access.

2. Create an event and generate five different invitations. Submit five legitimate test responses. Confirm the collector rejects an invalid signature and a replay of a used token.

3. While open, confirm exact response counts and comments stay hidden. Close the event, wait 15 minutes, then refresh. Confirm the eligible report releases and suppressed breakdowns stay hidden.

4. Restart the services. Verify the event report persists, comments keep their sealed order, used tokens stay spent, and a sealed real event cannot reopen. Administrator sign-in will be required again.

5. Add an action, move its status, reload, remove and undo. Confirm actions remain private to that browser. Download the brief and CSV; confirm no comments, codes or tokens appear. Review action JSON separately as private working material.

6. For event comparison, complete two real sealed events. Confirm the choices exclude synthetic demos and the displayed change matches the difference between rounded averages. Do not add fake events just to fill this view.

Operational responsibilities

Keep issuer keys available for existing events and restrict backup access. Define a club retention policy before real use; automatic scheduled deletion is not implemented. Back up both databases separately and practise restoring them. Keep Node and dependencies maintained. If a secret is exposed, rotate it through the hosting environment and review access logs using the minimum necessary metadata.

Source map

src/main.jsx owns routes, sign-in, events and submission. src/Front.jsx owns public pages. src/Dashboard.jsx and src/ReportStudio.jsx own reports and organiser tools. server/index.js is the collector; server/issuer.js is the issuer; server/store.js handles storage; server/privacy.js defines release rules. tests contains the automated checks. README.md contains developer setup instructions.

Hackathon demonstration and next steps

A focused four minute demonstration

Time	Show	Explain
0:00 to 0:30	Homepage and two entrances	Clubs need actionable feedback without attaching names to criticism.
0:30 to 1:20	Participant code exchange and form	A one-use invitation becomes an anonymous signed token. The code does not accompany feedback.
1:20 to 2:10	Administrator statistics and protected charts	Reports wait for closure, a minimum cohort and cooldown. Clearly identify synthetic data.
2:10 to 3:00	Listening note to action board	Add an action, choose a responsible role, set a deadline, and change its status.
3:00 to 3:30	Download brief and privacy explanation	Aggregate exports support club decisions without exporting individual comments.
3:30 to 4:00	Architecture and tested boundaries	Separate issuer and collector, replay prevention, access control and honest limitations.

Prepare before presenting

Keep one participant browser profile and one administrator profile ready. Generate a fresh invitation for the live demonstration. Have the PDF guide and a screenshot available offline. Use the five sample ratings only as a labelled synthetic example. Prepare a genuine sealed test report in advance if you want to show real report release; do not shorten the production cooldown for the presentation.

Answer likely judge questions

Why no participant password? The code proves event access without creating an identity record. Can organisers trace feedback? The protocol separates codes from tokens, but traffic metadata and identifying comments remain risks. Why are some charts locked? Small-group disclosure is intentionally limited. Is this AI analysis? No; current listening prompts use explicit keyword rules. Does it scale across clubs? The current configuration is one club with one admin credential pair; multi-tenant roles are future work.

Prioritised next work

Before club launch: complete live database and domain validation, mobile testing, backup restore and a retention policy. Next product release: shared action storage with role-based admin accounts, MFA and account recovery. Later research: privacy-preserving reply receipts, accessibility audit and an independent review of the end-to-end anonymity design. These items are not implemented features.

A strong presentation demonstrates one complete problem-solving journey and explains the trade-offs accurately. The code can support that demonstration; no feature list guarantees a competition result.